HiLTL: Progress-Conditioned Dual-Target Critics for Logic-Conditioned Reinforcement Learning

Anonymous Authors
Anonymous Institution

***Abstract*—Learning policies conditioned on temporal-logic specifications requires representing propositional constraints and logical progress. We study a critic that combines a one-step temporal-difference estimator with an episode-return estimator through an automaton-distance gate. HiLTL uses a guard-aware graph encoder, optional logical history, and auxiliary losses alongside proximal policy optimization. Fusion is an architectural inductive bias rather than an exact decomposition of satisfaction probability. We specify acceptance, reward, truncation, and optional temporally extended decision rules, distinguishing finite-task completion from bounded observations of recurrent specifications. An executable implementation has passed targeted semantic, gradient, persistence, and short-training checks. These establish limited implementation evidence; comparative performance and zero-shot generalization remain to be evaluated. We provide a controlled evaluation protocol separating encoder, critic, history, and option effects, without claiming unverified success-rate improvements.**

***Index Terms*—Linear temporal logic, reinforcement learning, automata, graph neural networks, value estimation.**

I. Introduction

Linear temporal logic (LTL) expresses temporally extended reachability, avoidance, and recurrence requirements [1, 2]. In logic-conditioned reinforcement learning (RL), an agent receives a specification and must relate its environment state to the remaining logical task.

Existing methods already provide mechanisms for representing and progressing specifications. LTL2Action conditions policies on formula structure and applies LTL progression [3]. GCRL-LTL composes goal-conditioned behaviors using automaton-based planning [4]. DeepLTL uses reach–avoid sequences, DeepSets, recurrent conditioning, and value-guided sequence selection [5]. Consequently, neither progress tracking nor automaton-based decomposition alone establishes a new contribution.

We investigate a narrower question: can automaton progress provide a useful inductive bias for combining critic estimators trained with different temporal targets? A bootstrapped target provides a local learning signal; an episode-return target incorporates observed outcomes over the remaining rollout. These estimators trade off bootstrapping error, sampling variance, and information restrictions. A progress-conditioned gate provides a structured mixture, whose benefit must be established against matched alternatives.

HiLTL couples this critic with a guard-aware automaton representation. Structural features describe transition roles, while guards retain the identity of their enabling conditions. Optional automaton history supplies trajectory context. A separate option extension uses real automaton transitions as candidate subgoals; it is not required by the primary critic.

Our methodological contributions are: (i) an explicit dual-target critic and PPO objective; (ii) guard-aware structural encoding and progress conditioning with specified boundary cases; and (iii) a controlled evaluation design separating representation, critic, and history changes. We report implementation checks, not improved task performance. The comparative experiments below remain outstanding.

II. Problem and Scope

A. Product Interaction and Acceptance

Let $\mathcal{M} = (S, A, P, \mu)$ describe an environment with unknown transition kernel, and let $\ell : S \to 2^{\mathsf{AP}}$ label states with atomic propositions. The alphabet is $\Sigma = 2^{\mathsf{AP}}$. A specification φ is translated into a transition-accepting limit-deterministic Büchi automaton

$$\mathcal{A}_\varphi = (Q, q_0, \Sigma, E, E_\epsilon, F_E).$$

A labeled edge $e = (q, g, q') \in E$ carries a propositional guard g, E_ϵ contains epsilon edges, and F_E contains accepting transitions. The labeled successor is determined by the current logical state and input label. Epsilon choices follow the converter and wrapper interface. Accepting transitions may be labeled or epsilon transitions. We retain the actual transition structure rather than imposing a desired state count.

A primitive action samples $s_{t+1} \sim P(\cdot \mid s_t, a_t)$ and the automaton consumes $\ell(s_{t+1})$. An enabled epsilon action changes logical state without advancing the underlying environment. Epsilon actions are masked where unavailable. The outer time-limit wrapper counts epsilon decisions toward H, although they do not advance physical simulation. Index t counts product decisions; $h_t = (q_0, \ldots, q_t)$ is logical history. The policy input is denoted $x_t = (s_t, q_t, h_t, \mathcal{A}_\varphi)$, omitting history in memoryless variants. Not every critic receives every component.

An infinite run is accepting if it traverses accepting transitions infinitely often. Merely visiting an accepting edge's target node is not an acceptance event: another incoming edge can be non-accepting. Reward and evaluation retain this distinction.

B. Optimized Surrogate

The implemented reward is

$$r_t = \mathbb{I}\{e_t \in F_E\} - \mathbb{I}\{\text{violation at } t\}. \tag{1}$$

Violation means entry into a bottom non-accepting strongly connected component (SCC). Environment termination is logged separately. This reward replaces the environment reward; it does not supply dense distance shaping.

Let $T \leq H$ be episode length under a fixed decision horizon H. Training optimizes

$$J_H(\pi) = \mathbb{E}_{\varphi\sim\mathcal{D}_{\text{train}},\,\tau\sim\pi}\left[\sum_{t=0}^{T-1}\gamma^t r_t\right], \qquad 0<\gamma<1. \quad (2)$$

For tasks certified by the wrapper's conservative finite-specification check, an accepting event terminates with success. Otherwise execution continues until the horizon or another termination. This check is not a complete characterization of co-safe LTL.

We do not assert equivalence between maximizing (2) and infinite-horizon LTL satisfaction probability. Such equivalence requires additional reward and MDP assumptions. Finite-task completion and bounded observations of recurrent or persistent specifications are evaluated separately: a finite trace cannot certify the infinite suffix required by GFp or FGp.

III. HiLTL

A. Guard-Aware Automaton Representation

Structural features. For features only, let $F_V = \{q' : (q,g,q') \in F_E\}$ be accepting-transition targets. Let $d(q)$ be directed hop distance to F_V along real edges, including epsilon edges. Set $D = \max(1, \max\{d(q) : d(q) < \infty\})$, taking the inner maximum as zero when empty. Define $\bar{d}(q) = d(q)/D$ when reachable and $\bar{d}(q) = 1$ otherwise. Node features are

$$u_q = [\mathbb{I}\{q \in F_V\}, \mathbb{I}\{q \text{ is cyclic}\}, \mathbb{I}\{q = q_0\}, \bar{d}(q)]. \quad (3)$$

Cyclic means membership in an SCC with at least two nodes or existence of a self-loop, not necessarily accepting recurrence. Distance to F_V ignores environment dynamics and is not a full characterization of Büchi acceptance.

Epsilon edges take precedence over progress edges ($d(q') < d(q)$); all others are non-progress. The four-position compatibility code is

$$c_e = \begin{cases} (0,0,0,1), & e \in E_\epsilon, \\ (1,0,0,0), & e \notin E_\epsilon,\ d(q') < d(q), \\ (0,1,0,0), & \text{otherwise.} \end{cases}$$

The third position is inactive, not a fourth observed type. The six structural edge features are

$$u_e = [c_e, \mathbb{I}\{q' \in F_V\}, \bar{d}(q')]. \quad (4)$$

For an edge with finite endpoint distances, $d(q) \leq 1 + d(q')$. A distance decrease greater than one is therefore impossible. We introduce neither artificial skip edges nor a skip-based contribution.

Guard encoding. For guard tokens $w_{e,1:L_e}$, learned embeddings and a GRU give

$$g_e = \text{GRU}(w_{e,1:L_e}), \qquad b_e = L_e^{-1}\sum_j \text{Emb}(w_{e,j}),$$
$$z_e = W_e[u_e \parallel g_e \parallel b_e] + b. \quad (5)$$

Tokens include Boolean operators, proposition identities in a fixed environment vocabulary, and a dedicated epsilon token. The mean pathway supplements the recurrent representation, but does not canonicalize logical expressions. The supported transfer setting retains proposition meanings.

Let $\mathcal{P}(q)$ contain outgoing accepting, epsilon, or distance-decreasing edges. Initial node embeddings include outgoing guard-conditioned values:

$$v_q^{(0)} = W_u u_q + b_u + \frac{\sum_{e\in\mathcal{P}(q)}\psi(z_e)}{\max(1, |\mathcal{P}(q)|)}, \quad (6)$$

where ψ is linear–LeakyReLU. Guard information thus enters message values rather than only normalized attention weights.

Message passing. Three GATv2 layers [6] use width 64 and four averaged heads. For a directed message $j \to i$,

$$a_{ij}^{(l)} = a_l^\top \,\text{LeakyReLU}(W_{i,l}v_i^{(l)} + W_{j,l}v_j^{(l)} + W_{e,l}z_{ji}),$$
$$\alpha_{ij}^{(l)} = \text{softmax}_{j\in\mathcal{N}^-(i)}\, a_{ij}^{(l)}. \quad (7)$$

Residual linear–LeakyReLU transformations and LayerNorm follow each convolution; no reverse edges or extra self-loops are added. Graph-wise softmax pooling gives $p^{(l)}$ at each layer. Single-scale variants use $p = p^{(3)}$; multi-scale variants concatenate all three pools.

Edge-aware attention is an established component, not itself our novelty. The design under study combines guard identity, structural features, and critic conditioning. Layer depth measures graph neighborhoods, not environment time. Global pooling does not give every node unrestricted global relational information. With fixed parameters the graph encoder is static: progress enters through the current-node query, history, and gate, not a state-dependent change in edge attention.

B. Progress-Conditioned Dual-Target Critic

Let $f(s_t)$ encode the environment and v_{q_t} be the current logical node embedding. Optional history is $z_t^{\text{hist}} = \text{LSTM}(v_{q_0}, \ldots, v_{q_t})$. The wrapper processes each product decision once. Immutable history snapshots are read, not advanced, during PPO replay.

The two estimators are

$$V_D(x_t) = \text{MLP}_D([f(s_t), p, v_{q_t}]),$$
$$V_R(x_t) = \text{MLP}_R([1-\bar{d}(q_t), v_{q_t}, z_t^{\text{hist}}]). \quad (8)$$

History is omitted when disabled. Subscripts D and R denote TD-supervised and return-supervised heads. V_R is an environment-agnostic regression proxy with restricted information, not a calibrated probability or, generally, the true environment-dependent value. Its notation indicates dependence through the displayed inputs only.

The primary gate and fused critic are

$$\beta_t = \sigma\big(3(1-\bar{d}(q_t) - \tfrac{1}{2})\big),$$
$$\widehat{V}_t = (1-\beta_t)V_D(x_t) + \beta_t V_R(x_t). \quad (9)$$

Here $\sigma(z) = (1+e^{-z})^{-1}$, giving $\beta_t \in [\sigma(-1.5), \sigma(1.5)] \approx [0.1824, 0.8176]$. This is a deterministic state-conditioned gate,

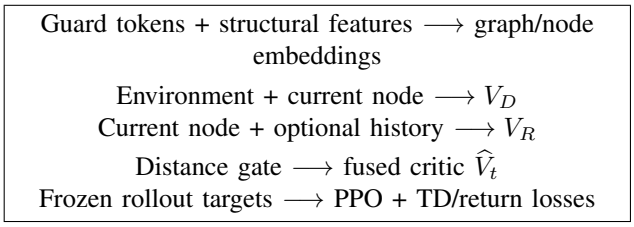


Fig. 1. Primary critic dependencies. The actor receives environment, graph, and current-node features, plus optional history. Optional subgoal conditioning is separate.

not a learned schedule. Constant $\beta = 1/2$ and a learned 64-hidden-unit MLP gate are separately trained alternatives, not interchangeable checkpoint settings.

Fusion defines an approximator, not a Bellman identity. A logical-state/history-only predictor can assign the same value to environment states with different future accessibility. Networks also omit an explicit remaining-horizon input, limiting exact representation of arbitrary finite-horizon values. These restrictions motivate matched empirical comparisons rather than a satisfaction guarantee.

C. Targets and PPO Optimization

Let $\bar{V}_t$ denote the fused prediction saved during collection, with $\bar{V}_T = 0$. For each complete episode, frozen targets are

$$y_t^D = r_t + \gamma \bar{V}_{t+1}, \qquad y_t^R = \sum_{k=t}^{T-1} \gamma^{k-t} r_k,$$
$$\delta_t = r_t + \gamma \bar{V}_{t+1} - \bar{V}_t, \qquad A_t = \delta_t + \gamma\lambda A_{t+1}, \tag{10}$$

with $A_T = 0$ and $\lambda = 0.95$. Termination and the defined horizon both use zero bootstrap; this is not continuing-task bootstrapping. Auxiliary losses are $L_D = \mathbb{E}[(V_D - \mathrm{sg}(y^D))^2]$ and $L_R = \mathbb{E}[(V_R - \mathrm{sg}(y^R))^2]$. The two heads concern the same external reward with different targets; they do not establish distinct physical time constants.

PPO [7] uses the fused critic for collection and advantages. The implementation standardizes advantages within the collected batch, clips them to $[-1, 1]$, and detaches them. For $\rho_t = \pi(a_t \mid x_t)/\pi_{\mathrm{old}}(a_t \mid x_t)$,

$$L_\pi = -\mathbb{E}[\min(\rho_t \widetilde{A}_t, \mathrm{clip}(\rho_t, 0.8, 1.2)\widetilde{A}_t)]. \tag{11}$$

The fused value loss L_V uses frozen targets $\bar{V}_t + A_t$ and the maximum of unclipped squared error and squared error after prediction clipping at $\bar{V}_t \pm 0.2$. The primitive objective is

$$L = L_\pi + 0.5L_V + L_D + 0.3L_R - 0.01\mathbb{E}[\mathcal{H}(\pi)]. \tag{12}$$

Single-head models omit the auxiliary losses. Adam uses learning rate 3×10^{-4} and global gradient clipping at 0.5. Targets are not differentiated through. Target correlation and information restrictions can cause interference or bias; improved learning is an empirical hypothesis.

D. Optional Transition-Targeted Options

An option [8] selects a real outgoing edge as its subgoal; initiation requires the edge's source logical state. Candidates exclude edges into bottom non-accepting SCCs. Non-accepting self-loops are removed when a state-changing, accepting, or epsilon alternative exists. A meta-policy scores target-node/edge embeddings. A shared primitive actor conditions on the selected embedding rather than a fixed global state ID.

An option terminates when its selected guard and source–target transition are realized, another logical state is reached, the duration limit is reached, or the episode ends. Epsilon subgoals require an actual epsilon decision. Recurrent self-loop realization is checked by its guard, not just equality of source and target. Let $d_e(q)$ be distance to the selected target capped at $|Q|$, and $\Phi_e(q) = -d_e(q)/\max(1, |Q|)$. The intrinsic reward is

$$r_t^{\mathrm{int}} = \mathbb{I}\{\text{selected edge realized}\} + 0.1(\Phi_e(q_{t+1}) - \Phi_e(q_t)). \tag{13}$$

This auxiliary signal omits the discount factor in its potential difference; we do not invoke discounted shaping invariance.

The primitive actor adds separately standardized, clipped intrinsic GAE to its external advantage. An option critic learns intrinsic segment returns, adding $0.5L_{\mathrm{int}}$ to (12). For duration k_n, the meta reward is $R_n = \sum_{j=0}^{k_n-1} \gamma^j r_{t_n+j}$. SMDP bootstrapping uses γ^{k_n} and advantage traces use $(\gamma\lambda)^{k_n}$. A separate meta PPO update uses the fused external critic, advantages clipped to $[-10, 10]$, value coefficient 0.5, and entropy coefficient 0.01. Primitive and meta updates share an optimizer and representations. Extra rewards and optimization make Options a separate intervention, not a pure encoder ablation. The design is distinguished from future-dependent options [9], without presuming comparative superiority.

IV. Analysis and Limitations

Proposition 1 (Conditional fusion error)**.** *For a reference finite-horizon value v_t^π, if $|V_D - v_t^\pi| \le \epsilon_D$ and $|V_R - v_t^\pi| \le \epsilon_R$ at an input, then*

$$|\widehat{V}_t - v_t^\pi| \le (1 - \beta_t)\epsilon_D + \beta_t \epsilon_R.$$

Proof. Substitute (9) and apply the triangle inequality with $0 \le \beta_t \le 1$. □

This elementary bound does not guarantee small errors, improvement over either head, or optimization convergence. An inaccurate return head can degrade fusion. If V_D were the exact Bellman backup of the true value, it would already equal that value; imposing an exact nontrivial mixture would then force V_R to equal it too. We instead specify two trained regressors.

Guard identity addresses a different limitation. Two isomorphic task automata can differ only in whether a transition requires blue or green. A permutation-invariant encoder receiving identical structural features but no guards produces identical task representations. At the same environment observation it cannot select different actions solely because of those

guards. This motivates guard-aware encodings, but does not prove the GRU/mean design optimal or sufficient for arbitrary Boolean reasoning.

SCC and distance preprocessing costs $O(|Q|+|E|)$, excluding conversion and guard processing. At fixed width and heads, message passing is linear in edges; node transformations have quadratic dependence on width. Guard cost depends on token length. Re-encoding full logical history at every decision can accumulate quadratic work in episode length. Graph complexity therefore does not determine end-to-end runtime: conversion, reference sequence search, simulation, and optimization must also be measured.

V. Evaluation Design and Available Evidence

A. Research Questions and Fair Comparisons

The evaluation asks: (RQ1) does dual-target estimation improve learning against a matched single critic; (RQ2) what do history and the gate contribute; (RQ3) do guard/structural encoding and multi-scale pooling improve held-out performance; and (RQ4) what are the benefits and computational costs of Options?

Required controls. The primary critic study must hold the environment encoder, graph encoder, actor, reward, task distribution, and budget fixed. Its factorial controls are single critic, dual-target critic, single critic with history, and dual-target critic with history. Gate studies independently retrain fixed, constant, and learned variants. When feature dimensions change, report parameter counts and a capacity-matched alternative. Guard-only versus additional structural features, and last-layer versus multi-layer pooling, isolate representation interventions. Removing all guards is a task-identifiability diagnostic, not a fair default baseline.

Existing configurations. Seven named implementation configurations are available. The sequence reference uses DeepSets/GRU, whereas every non-reference configuration uses the guard-aware GNN. The multi-scale switch changes pooled layers, not whether a GNN exists. The history switch controls an LSTM; current logical state remains observed in every product model. Consequently, the sequence-reference versus HV-only comparison changes the whole architecture and cannot isolate HV. Matched single-GNN-critic and history-only controls are required additions, not completed experiments. Options are compared with the otherwise identical full model without Options.

External methods. DeepLTL [5] is not a static GNN baseline. The repository's reference architecture and product-training adapter are not a bit-identical reproduction of its upstream method or curriculum. LTL2Action [3] and GCRL-LTL [4] have not been rerun here. Published values are historical context, not inputs to new statistical comparisons. DeepLTL's published evaluation used 15M interactions and different specifications; those values cannot serve as measurements on our proposed 1M-interaction suite. We make no same-protocol numerical superiority claim over these methods.

TABLE I
Diagnostic task split; formal evaluation requires expansion and independent validation. Here b, g, y denote blue, green, yellow.

Split	Formulas
Train	$Fb \wedge Fg$; Fy; $F(b \wedge Fg)$
Held-out	$GFb \wedge GFy$; $F(b \wedge F(g \wedge Fy))$; $(\neg b\ U\ g) \wedge Fy$

B. Task Distribution and Held-Out Evaluation

Target environments are PointLtl2-v0, LetterEnv-v0, and FlatWorld-v0. Models train separately in each environment: this is multi-environment evaluation, not cross-environment transfer. Zero-shot evaluation uses new formulas in the same environment and proposition vocabulary without weight updates or test-guided checkpoint selection.

A full study requires fixed training, validation, and test manifests covering reachability, avoidance, ordering, and recurrence. Validation determines hyperparameters and checkpoint-selection rules before test evaluation. Formula length, actual automaton size, edge count, and diameter provide structural strata; graph size alone does not measure environment difficulty. Logical satisfiability and realizability under environment constraints must be considered separately. Language-equivalent formulas must not knowingly cross splits.

Current engineering manifests contain three training and three held-out formulas per environment, without independent validation. Table I gives the color-domain lists; LetterEnv substitutes a, b, c. These small lists are diagnostic, not the finalized benchmark. These manifests do not define a broad benchmark: additional independent formulas and validation tasks must be fixed before the comparative study.

The loader rejects whitespace-normalized duplicates and overlapping fingerprints of raw and environment-pruned, guard/acceptance-labeled automata. State-number-independent graph hashing is a safeguard, not a complete language-equivalence procedure. A zero-shot checkpoint must never train on its test formulas, even if those formulas appear in a separate task-specific study. Split assignments and checkpoint provenance must remain explicit.

C. Training, Metrics, and Statistical Units

The proposed starting budget is 1M interactions per independent model, with training seeds 1–5. This is a protocol choice, not an accomplished run. Current defaults are $\gamma = 0.998$, rollout threshold 512, minibatch size 64, and four PPO epochs. Complete episodes are collected until the threshold is reached; actual interactions can exceed requested budgets and must be reported. Horizons, maps, reset distributions, action/counting conventions, and selected checkpoints must be fixed in the finalized manifest. Short-run timing is not a reliable full-training estimate.

For finite tasks report completion within H, violation rate, all-episode length, and successful-episode length with its success count. Do not silently substitute one step statistic for another. For recurrent tasks report accepting-transition

TABLE II
RECORDED IMPLEMENTATION CHECKS. CASES ARE NOT INDEPENDENT PERFORMANCE TRIALS OR TRAINING REPLICATIONS.

Check group	Cases
Graph/value/trajectory semantic regressions	12
Options, targets, split, reference checks	8
Final CLI short-run matrix, including expected rejections	34
Independent-process reference/option restoration	2

counts and a defined rate under a common horizon. Where the automaton permits interpreting these as completed goal cycles, state the relationship explicitly. Do not average them with finite SR or label them infinite satisfaction probabilities. Any finite-trace persistence proxy needs a predeclared event. Reward and discounted return refer to (1).

For each training seed and test formula, the proposed evaluation uses 500 episodes with separately specified reset seeds. Deterministic and sampled policies are distinct protocols. Average episodes within each independent training seed, then report seed-level means, sample standard deviations, and 95% Student-t intervals. Paired differences use matched seed indices and task conditions. Episodes are not independent training repetitions. Formula/environment macro-averaging must be fixed in advance; uncertainty over sampled formulas differs from training-seed uncertainty. One training seed supplies no estimate of between-training variability.

Absolute SR differences are percentage points; relative change is $(\mathrm{SR}_{\mathrm{new}} - \mathrm{SR}_{\mathrm{ref}})/\mathrm{SR}_{\mathrm{ref}}$ for positive reference SR. Intervals must target the same effect quantity. Qualitative significance labels and subtraction of rounded relative improvements are avoided. Checkpoints, split hashes, actual interactions, episode outcomes, and analysis inputs are retained for reconstruction.

D. Implementation Evidence and Outstanding Experiments

Targeted implementation checks are summarized in Table II. They concern selected inputs and short executions, not every supported trace. The final short-run matrix exercised seven configurations, held-out evaluation and strict restoration, gate variants, Letter/Zone option runs, and a small CUDA run. FlatWorld sequence-reference and option checkpoints were also restored in independent processes. Frozen-weight evaluation and seed-level aggregation were checked, including rejection of repeated training seeds and invalid episode counts.

Checks distinguishing guard identities motivated direct message values and the mean-token pathway. Options checks verify actual parameter updates rather than only an interface. This supports the intended data flow on tested cases, not an SR advantage, faster convergence, robust generalization, or correctness for every LTL formula.

Matched critic controls, independent validation, an expanded task set, full-budget multi-seed curves, and formal 500-episode evaluations remain outstanding. No comparative performance table is reported before these experiments. No illustrative performance or gate plot is used as empirical evidence. The final study must retain failed runs and negative results, including possible degradation from fusion, multi-scale features, or Options.

VI. RELATED WORK

Logic-conditioned methods differ in representation and planning. LTL2Action uses formula structure and progression [3]; GCRL-LTL combines proposition-conditioned skills with automaton planning [4]; DeepLTL reasons over reach–avoid sequences with recurrent conditioning [5]. Reward machines formalize structured non-Markovian rewards [10]. Neither static representation nor absent progress tracking describes all these approaches.

Graph attention [6, 11] and message passing [12] motivate the encoder, but edge features and stacked layers are established tools. Our representation question concerns guard identity and structural progress, not inventing edge-aware attention. Similarly, different regression targets require simpler multi-head and capacity-matched critic comparisons before being credited with improved temporal reasoning.

Options formalize temporally extended actions [8]; Option-Critic learns option policies and termination [13]; future-dependent options study temporal-logic task generalization [9]. Our optional controller specifies candidate generation, guard-based termination, and SMDP updates without claiming superiority to these alternatives. Evaluation must expose training-seed variability and implementation differences [14].

VII. CONCLUSION

HiLTL studies progress-conditioned fusion of TD- and return-supervised critics with guard-aware logical representations. The formulation distinguishes approximate return estimation from satisfaction probability and specifies history, finite-horizon targets, and optional temporally extended decisions. Available checks establish limited implementation evidence. Improved sample efficiency, robustness, and held-out formula performance remain empirical questions for matched multi-seed experiments. No performance gain or general satisfaction guarantee follows from the present evidence.

APPENDIX A
REPRODUCIBILITY AND OPERATIONAL DEFINITIONS

The implementation is identified as `hiltl-product-v2`. Graph models flatten observations into a two-layer 128/64 environment MLP; the TD and return heads have two hidden layers of width 256 and 128, respectively. History uses a 64-unit LSTM. The sequence reference uses environment-specific DeepLTL components, an encoder difference that cannot be hidden in a critic ablation.

The repository's Rabinizer4 pipeline produces automata, which are pruned using possible environment assignments, sink-completed when necessary, and analyzed for SCCs. State counts depend on conversion and pruning; they are recorded

from the generated graph, not assigned from a formula difficulty label. Formal experiments must preserve conversion options, actual graphs, and node/edge counts.

Option duration defaults to 50 decisions. Intrinsic returns bootstrap to zero at segment boundaries; meta targets use external SMDP returns. Primitive and meta updates share representations, so their additional computation must enter runtime comparisons.

Rollouts store logical state, immutable history, behavior log-probability, value, and selected option where applicable. The sequence reference stores the sequence chosen during collection; PPO replay does not reselect it. Restoration validates model configuration, task manifest, and implementation identity. Resumption starts a new environment episode, without promising an uninterrupted trajectory. Evaluation verifies unchanged model weights.

Formal evidence must preserve per-seed checkpoints, versions, manifests, actual budgets, all evaluation outcomes, and statistical inputs. Predetermined budget points supply learning curves; validation-only rules select checkpoints. End-to-end cost includes conversion, encoding, simulation, optimization, and checkpointing, with hardware and concurrent load disclosed. A layer microbenchmark or short GPU run does not establish practical acceleration.

References

[1] A. Pnueli, "The temporal logic of programs," in *18th annual symposium on foundations of computer science (sfcs 1977)*. ieee, 1977, pp. 46–57.

[2] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT press, 2008.

[3] P. Vaezipoor, A. C. Li, R. Toro Icarte, and S. A. McIlraith, "LTL2Action: Generalizing LTL instructions for multi-task RL," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10 497–10 508.

[4] W. Qiu, W. Mao, and H. Zhu, "Instructing goal-conditioned reinforcement learning agents with temporal logic objectives," in *Advances in Neural Information Processing Systems*, 2023.

[5] M. Jackermeier and A. Abate, "DeepLTL: Learning to efficiently satisfy complex LTL specifications for multi-task RL," in *International Conference on Learning Representations (ICLR)*, 2025.

[6] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv preprint arXiv:1707.06347.

[8] R. S. Sutton, D. Precup, and S. Singh, "Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning," *Artificial intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.

[9] D. Xu and F. Fekri, "Generalization of temporal logic tasks via future dependent options," *Machine Learning*, 2024.

[10] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Reward machines: Exploiting reward function structure in reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022.

[11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[12] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *International conference on machine learning*. Pmlr, 2017, pp. 1263–1272.

[13] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 31, no. 1, 2017.

[14] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.